End - End Communication at Transport layer

a) Implementation echo client server using TCP Sockets

Aim:

To implement an Echo client - Server application using TCP Sockets and unity communication between the client and Server.

Procedure:

* To create echat Server program using Socket programming
* Start and run the chat Server
* Create the chat client program and connect to the server
* Run two or more client program in separate terminals
* Enter a name for each client.
* Send the messages from client to Server.
* The Server broadCasts the message to the other clients
* Verify that the clients can exchange message through the Server.
* Continue the communication between the client.
* Enter / quite to terminate a client connection.

Program:

```
import java.io.*;
import java.net.*;

Public class TCPEchoServer{
    Public static void main (String[] args) throws Exception{
        ServerSocket ss= new ServerSocket (5000);
        Socket s=ss.accept();

        BufferedReader in = new BufferedReader (
```

```
Socket s = ss.accept();
BufferedReader in = new BufferedReader(
    new InputStreamReader(s.getInputStream()));
PrintWriter out = new PrintWriter(s.getOutputStream(),
                                  true);
String msg;
while ((msg = in.readLine()) != null) {
    out.println("Echo: " + msg);
    if (msg.equalsIgnoreCase("bye")) break;
}
s.close();
ss.close();
}
}
import java.io.*;
import java.net.*;
class Client {
public static void main (String[] args) throws Exception {
Socket s = new Socket("localhost", 5000);
BufferedReader in = new BufferedReader(
    new InputStreamReader(s.getInputStream()));
PrintWriter out = new PrintWriter(s.getOutputStream(),true);
BufferedReader kb = new BufferedReader(
    new InputStreamReader(System.in));
String msg;
while ((msg = kb.readLine()).equalsIgnoreCase("bye")) {
out.println(msg);
System.out.println(in.readLine());
}
s.close();
}
```

Result:

Thus, the TCP Echo client server application was successfully implemented. Client successfully received the entered message from the server.

b) Implement Echo client server using UDP Socket

Aim:

To implement an echo client server application using UDP Socket and verify - Communication between the client and server.

Procedure:

1) Create a UDP Echo Server using DatagramSocket and DatagramPacket
2) Assign a port no. to the UDP server
3) Compile and execute the UDP Server program.
4) Create a UDP Echo client using DatagramSocket and DatagramPacket
5) Specify the Server Address and port no.
6) Enter a message through the client
7) The client sends the message as a UDP datagram to Server
8) The server receives the datagram and sends
9) Display the received messages
10) Enter bye to terminate the communication.

Program:

```
import java.net.*;
Public class UDPEchoServer {
    public static void main (String[] args) throws Exception {
    DatagramSocket s = new DatagramSocket (666);
    byte[] b = new byte[1024];
    while (true) {
        DatagramPacket p = new DatagramPacket (b, b.length);
```

```
S Writer (f);
String msg = new String(p.getData(), 0, p.getLength());
byte[]x = ("Echo: " + msg).getBytes();
S.send(new DatagramPacket(x, x.length, p.getAddress(), p.getPort()));
if (msg.equalsIgnoreCase("bye")) break;
}
S.close();
}
}

import java.net.*;
import java.io.*;
public class DatagramClient{
  public static void main(String args[])throws Exception{
  DatagramSocket s = new DatagramSocket();
  InetAddress ia = InetAddress.getByName("local host");
  BufferedReader br= new BufferedReader(
     new InputStreamReader(System.in));
  String msg;
  while(!(msg = br.readLine()).equalsIgnoreCase("bye")){
  byte[] b = msg.getBytes();
  s.send(new DatagramPacket(b, b.length, ia, 600));
  byte[]r = new byte[1024];
  s.receive(p);
  System.out.println(new String(p.getData(),0,p.getLength()));
}
  s.close();
}
```

Result:

Thus the UDP Echo Client - Server application successfully implement, and the Client received message from the Server

3) To implement a chat client - server application using Sockets.

Aim:

To implement a client - server application using Socket which enables communication between multiple clients through server.

Procedure:

1) Create a chat server program using Socket Programming
2) Start and run the chat server.
3) Create the chat program and connect it to the server.
4) Run two or more chat program in separate terminals.
5) Enter a name for each client.
6) Send messages from client to server.
7) The server broadcast the message to the connected client that identify that the client can exchange messages through server.
8) Enter !quit to terminate client connection.

Program:

```
Import java.io.*;
Import java.net.*;
Import java.util.*;

Public class chat server {
    Static Set<PrintWriter> writers = new HashSet<>();

    Public static void main(String[] args) throws Exception {
        ServerSocket s = new ServerSocket(3000);
        System.out.println("Chat Server Started");
```

```
PS C:\Users\DELL\Downloads\cn> javac ChatClient.java
PS C:\Users\DELL\Downloads\cn> java ChatClient
Enter your name:
nithish
Welcome, nithish! Type /quit to exit.
```

```
        at java.base/java.net.ServerSocket.<init>(ServerSocket.java:239)
        at ChatServer.main(ChatServer.java:11)
PS C:\Users\DELL\Downloads\cn> netstat -ano | findstr :5000
  TCP    0.0.0.0:5000           0.0.0.0:0              LISTENING       12344
  TCP    127.0.0.1:5000         127.0.0.1:50543        ESTABLISHED     12344
  TCP    127.0.0.1:50543        127.0.0.1:5000         ESTABLISHED     12344
  TCP    [::]:5000              [::]:0                 LISTENING       12344
PS C:\Users\DELL\Downloads\cn> taskkill /PID 12344 /F
SUCCESS: The process with PID 12344 has been terminated.
PS C:\Users\DELL\Downloads\cn> java ChatServer
Chat server started on port 5000
nithish connected from /127.0.0.1
```

```
while (true){
    Socket p = s.accept();
    newThread(() -> chat(p)).start();
}
}
static void chat(Socket s){
  try{
    BufferedReader in = new BufferedReader(
    new InputStreamReader(s.getInputStream()));
    PrintWriter out = new PrintWriter(s.getOutputStream(),
    true, "cp437");
    String msg;
    while ((msg = in.readLine()) != null){
      for (PrintWriter p : c) p.println(msg);
      c.remove(out);
      s.close();
    } catch (Exception e){}
    }
}.

import java.io.*;
import java.net.*;
Public class ChatClient{
  Public static void main(String[] a) throws IOException{
    Socket s = new Socket("127.0.0.1", 6666);
    BufferedReader in = new BufferedReader(new
                    InputStreamReader(s.getInputStream()));
```

```
newThread (() ->{
    try {
        String m;
        While ((m = in.readline()) != null) System.out.
                                            println(m);
        Catch (Exception e) {}
}).start();

String m;
While ((m = kb.readline()) != null){
    out.println(m);
    if(m.equalIgnore("quit")) break;
}
s.close();
}
}
```

Result:-

Thus, the Chat Client Server application was successfully implemented, and messages were successfully exchanged between multiple client